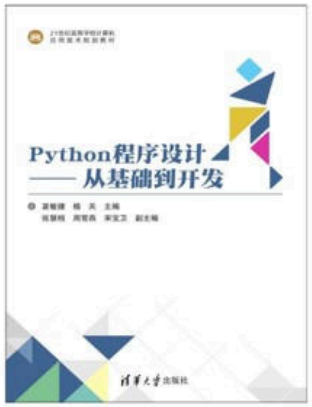
東北大學 国家示范性软件学院
SOFTWARE COLLEGE OF NORTHEASTERN UNIVERSITY

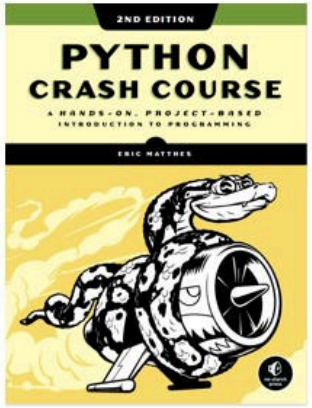


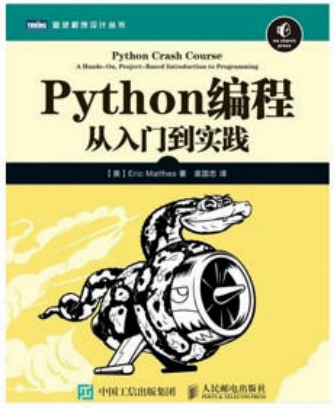
Programming in Python

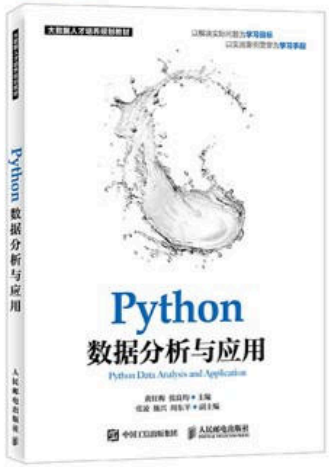
—— PART I Basics

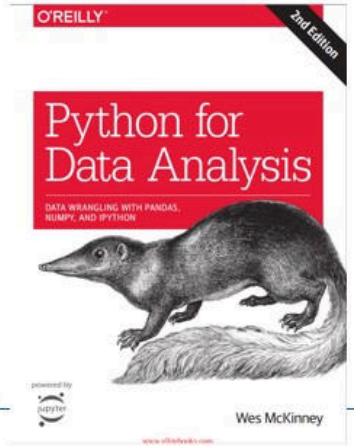
Reference Books













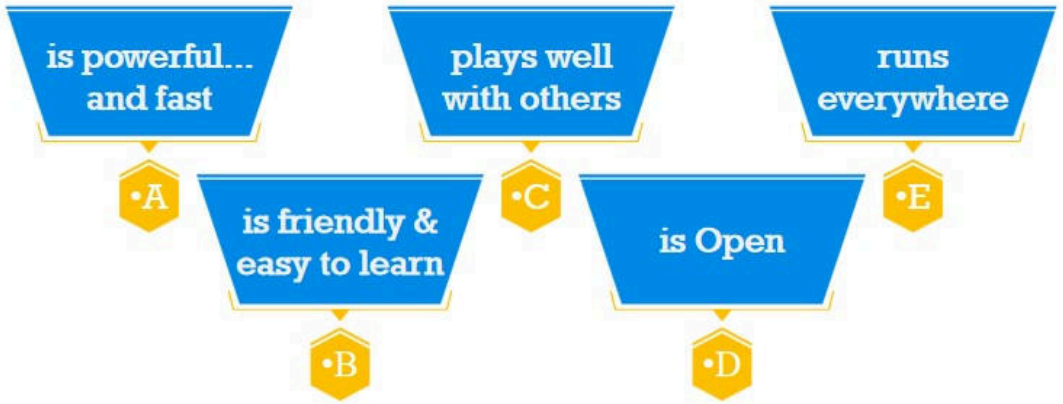
Chapter 1: Getting Started

Contents

1. Why Python?
2. Setting Up Anaconda
3. Jupyter Notebook
4. Python: Basic input/output
5. Python Grammar specification (代码规范)

Why Python?





- Python is an incredibly **efficient** language: your programs will do more in fewer lines of code than many other languages would require.
- Python's syntax will also help you write "**clean**" code. Your code will be easy to read, easy to debug, and easy to extend and build upon compared to other languages.

Why Python?



TIOBE Index for Aug 2026

Aug 2026	Aug 2025	Change	Programming Language	Ratings	Change
1	1		Python	18.53%	-7.61%
2	3	↑	C	11.10%	+2.07%
3	2	↓	C++	8.62%	-0.56%
4	4		Java	8.25%	-0.34%
5	5		C#	4.09%	-1.43%
6	6		JavaScript	2.63%	-0.52%
7	7		Visual Basic	2.18%	-0.15%
8	12	⇈	SQL	1.88%	+0.16%
9	14	⇈	R	1.56%	+0.19%
10	18	⇈	Rust	1.45%	+0.31%

Why Python?

Python语言特点



- 是一种面向对象、**解释型**计算机程序设计语言；
- 纯粹的**自由软件**；
- Python**语法简洁清晰**，用空格(space)作为语句缩进；
- Python具有**丰富和强大的库**。它常被昵称为胶水语言，能够把用其他语言制作的各种模块（尤其是C/C++）很轻松地联结在一起；
- Python**快速生成**程序的原型；
- 封装Python可以调用的**扩展类库**；
- **跨平台**，程序无需修改在Windows、Linux、Unix、Mac等操作系统上使用

雨课堂 Rain Classroom

Why Python?

People use Python for many purposes

- to make games
- build web applications
- solve business problems
- develop internal tools at all kinds of interesting companies
- used heavily in scientific fields for academic research and applied work.

需要什么应用就能找到什么库

雨课堂 Rain Classroom

Why Python?

Disadvantages of Python:

- The running speed is slow;
- The code cannot be encrypted加密;
- The way of distinguishing statement relationships by indentation缩进 still brings confusion to many beginners

雨课堂 Rain Classroom

Setting Up Anaconda

- 初学一门编程语言，最核心、普遍最难的不是语法，而是这几层障碍：

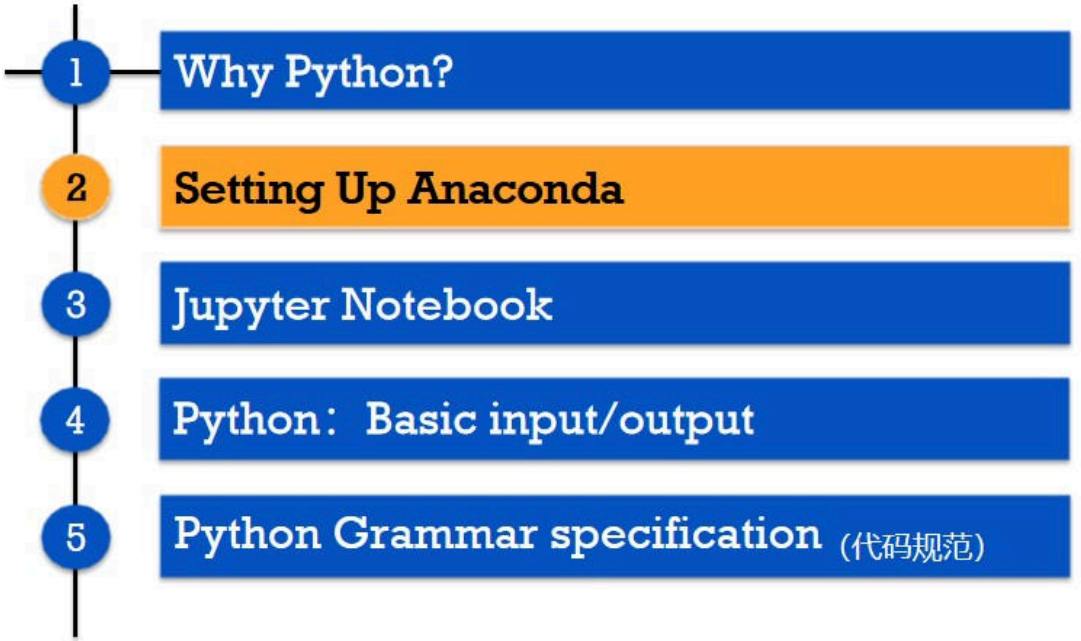
1. **入门前 3 周**（语法期）：**环境配置** + 机器思维转换（最难）
   - 安装、版本冲突、依赖、路径、编辑器配置。还没开始学语法，先被环境报错卡住很久，很多人在这里直接放弃。

2. **中期**（写小项目）：模块化抽象、调试能力
3. **后期工程化**：规范、依赖管理、性能、协作

雨课堂 Rain Classroom

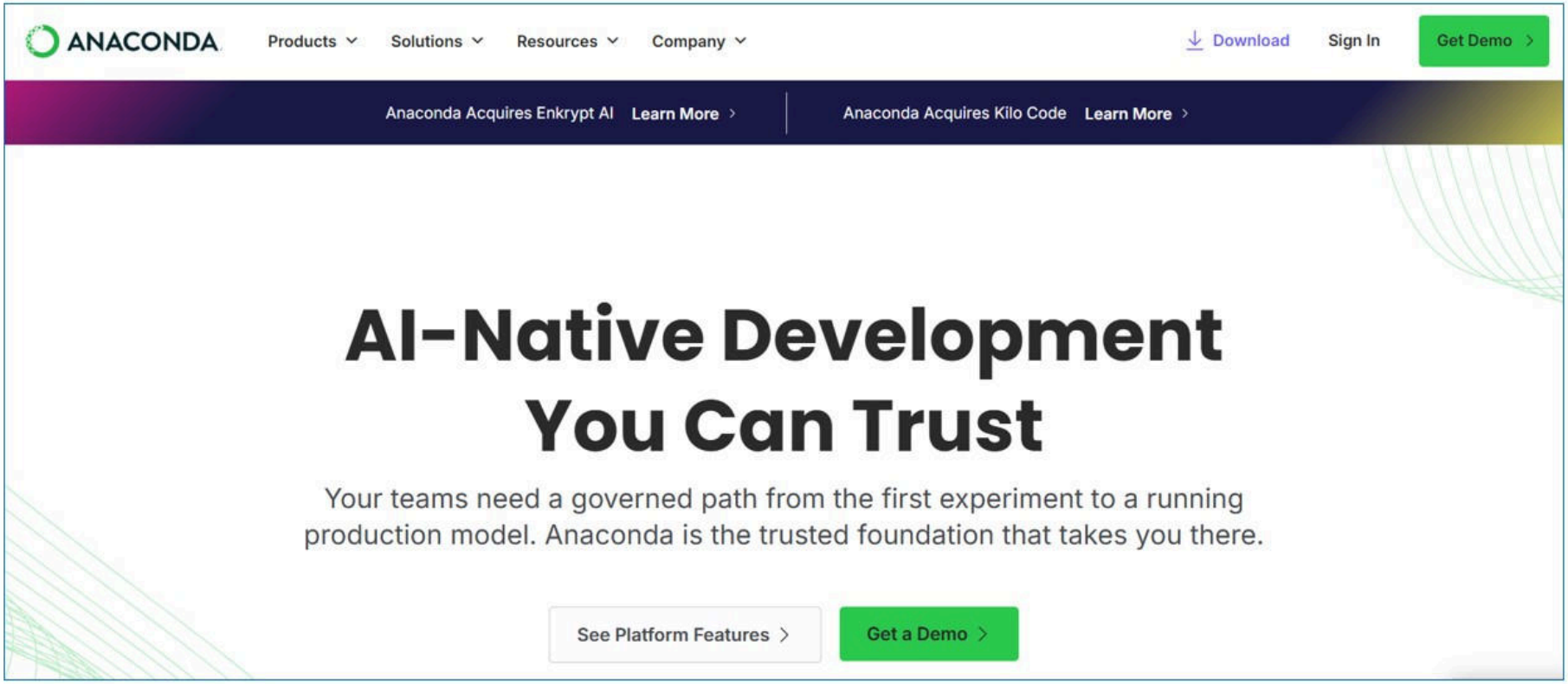
Chapter 1: Getting Started

Contents



Setting Up Anaconda Distribution

https://www.anaconda.com/



Setting Up Anaconda Distribution

https://www.anaconda.com/

- Anaconda offers the easiest way to perform Python/R data science and machine learning on a single machine.
- Start working with thousands of open-source packages and libraries today.

Setting Up Anaconda Distribution

Installing Additional Python Packages

- Python 3.4+ includes **pip** and **setuptools**
  - **pip**: to install and manage Python extension packages
  - **setuptools** for publishing Python packages

- E.g

```
> python -m pip install -upgrade pip setuptools          # 更新 pip 包和 setuptools 包
> pip install numpy-1.21.2+mkl-cp39-cp39-win_amd64.whl   #安装 NumPy 包，得先下载相应NumPy版本
> pip install -U matplotlib                              #在线安装 Matplotlib 包，可能需要先安装其他依赖包
```


雨课堂 Rain Classroom

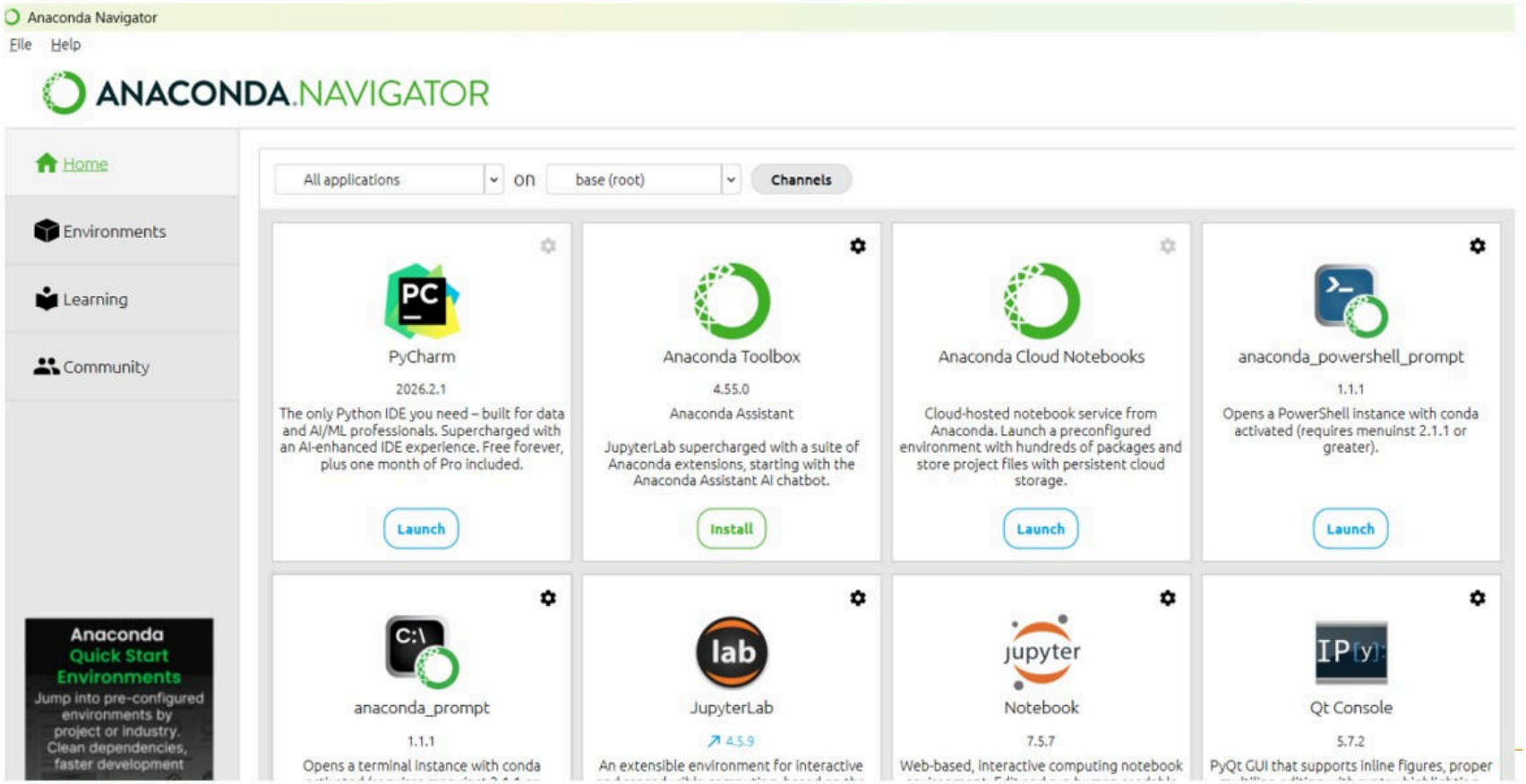
Setting Up Anaconda Distribution

The most popular open-source Python distribution platform

- Anaconda Distribution contains conda and Anaconda Navigator.
- Python and hundreds of scientific packages are included with the Anaconda installation.
- Anaconda包含了conda、Python在内的超过180个科学包及其依赖项。
- 完全开源和免费
- 额外的加速和优化是收费的，但对于学术用途，可以申请免费的 License
- 对全平台和几乎所有Python版本支持

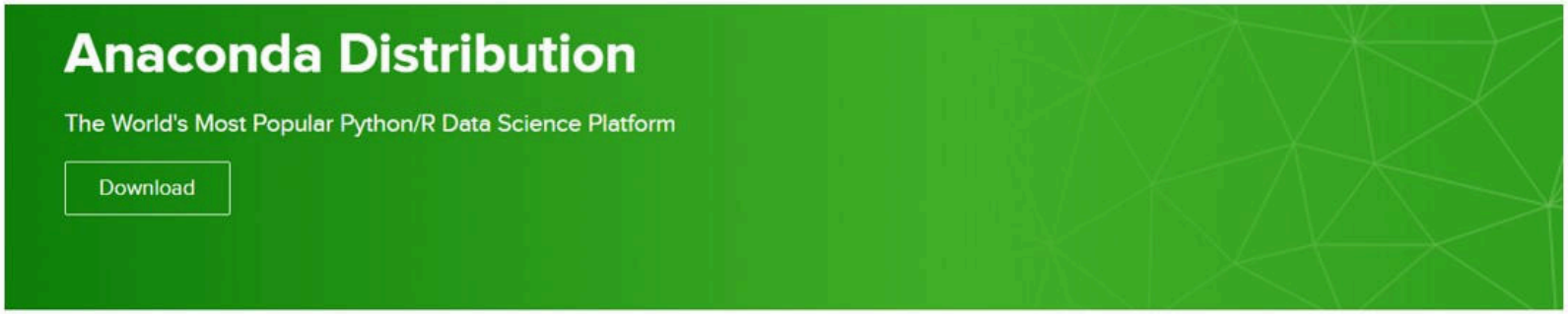
雨课堂 Rain Classroom

Setting Up Anaconda Distribution



Setting Up Anaconda Distribution

https://www.anaconda.com/

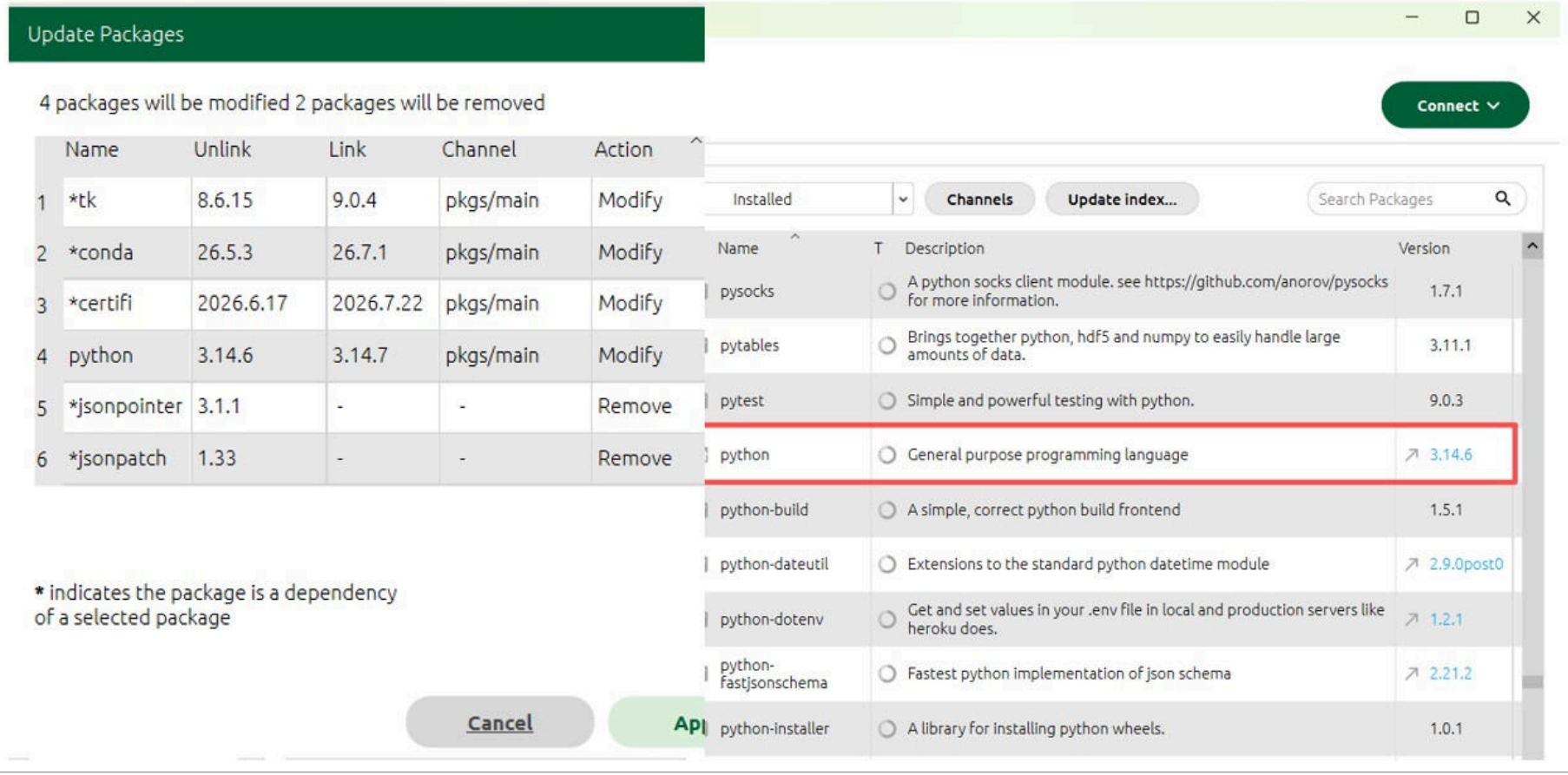


The open-source Anaconda Distribution is the easiest way to perform Python/R data science and machine learning on Linux, Windows, and Mac OS X. With over 15 million users worldwide, it is the industry standard for developing, testing, and training on a single machine, enabling *individual data scientists* to:

- Quickly download 1,500+ Python/R data science packages
- Manage libraries, dependencies, and environments with Conda
- Develop and train machine learning and deep learning models with scikit-learn, TensorFlow, and Theano
- Analyze data with scalability and performance with Dask, NumPy, pandas, and Numba
- Visualize results with Matplotlib, Bokeh, Datashader, and Holoviews



Setting Up Anaconda Distribution



Setting Up Anaconda Distribution

1. installation platforms

- Windows
- macOS
- Linux

2. installation conditions

- System Requirements: 64-bit
- Download file size: 900MB
- Required space size: 3GB

雨课堂 Rain Classroom

Anaconda、conda、pip 的区别

① Anaconda

- Anaconda是一个包含180+科学包及其依赖项的发行版本。
- 其包含的科学包有：conda, numpy, scipy, ipython notebook等。

雨课堂 Rain Classroom

Anaconda、conda、pip 的区别

② conda

- conda是包及其依赖项和环境的管理工具
  - 快速安装、运行和升级包及其依赖项
  - 在计算机中便捷地创建、保存、加载和切换环境
- 适用语言：Python, R, Ruby, Lua, Scala, Java, JavaScript, C/C++, FORTRAN
  - conda为Python项目而创造，但可适用于上述的多种语言
- 适用平台：Windows, macOS, Linux
- conda包和环境管理器包含于Anaconda的所有版本当中。

雨课堂 Rain Classroom

Anaconda、conda、pip 的区别

③ pip

- pip是用于安装和管理软件包的包管理器
- pip编写语言：Python
- pip是Python中默认安装的：
  - ✓ Python 2.7.9及后续版本：命令为pip
  - ✓ Python 3.4及后续版本：命令为pip3

雨课堂 Rain Classroom

Anaconda、conda、pip 的区别

pip 与 conda 比较

- **依赖项检查**
  - pip：
    - ✓不一定会展示所需其他依赖包。
    - ✓安装包时或许会直接忽略依赖项而安装，仅在结果中提示错误。
  - conda：
    - ✓列出所需其他依赖包。
    - ✓安装包时自动安装其依赖项。
    - ✓可以便捷地在包的不同版本中自由切换。

Anaconda、conda、pip 的区别

pip 与 conda 比较

- **环境管理**
  - pip：维护多个环境难度较大。
  - conda：比较方便地在不同环境之间进行切换，环境管理较为简单。
- **对系统自带Python的影响**
  - pip：在系统自带Python中包的**更新/回退版本/卸载将影响其他程序。
  - conda：不会影响系统自带Python。
- **适用语言**
  - pip：仅适用于Python。
  - conda：适用于Python, R, Ruby, Lua, Scala, Java, JavaScript, C/C++, FORTRAN。23

雨课堂 Rain Classroom

Setting Up Anaconda Distribution

- If it is already installed, but not the latest version you can:
  - Automatically detect and upgrade all upgradeable libraries in the Anaconda manager:

    > conda update --all

  - update anaconda itself

    > conda update anaconda

雨课堂 Rain Classroom

Using conda from Anaconda

- From the command-line
  - Windows: "Anaconda Prompt"
  - macOS/Linux: "Terminal"

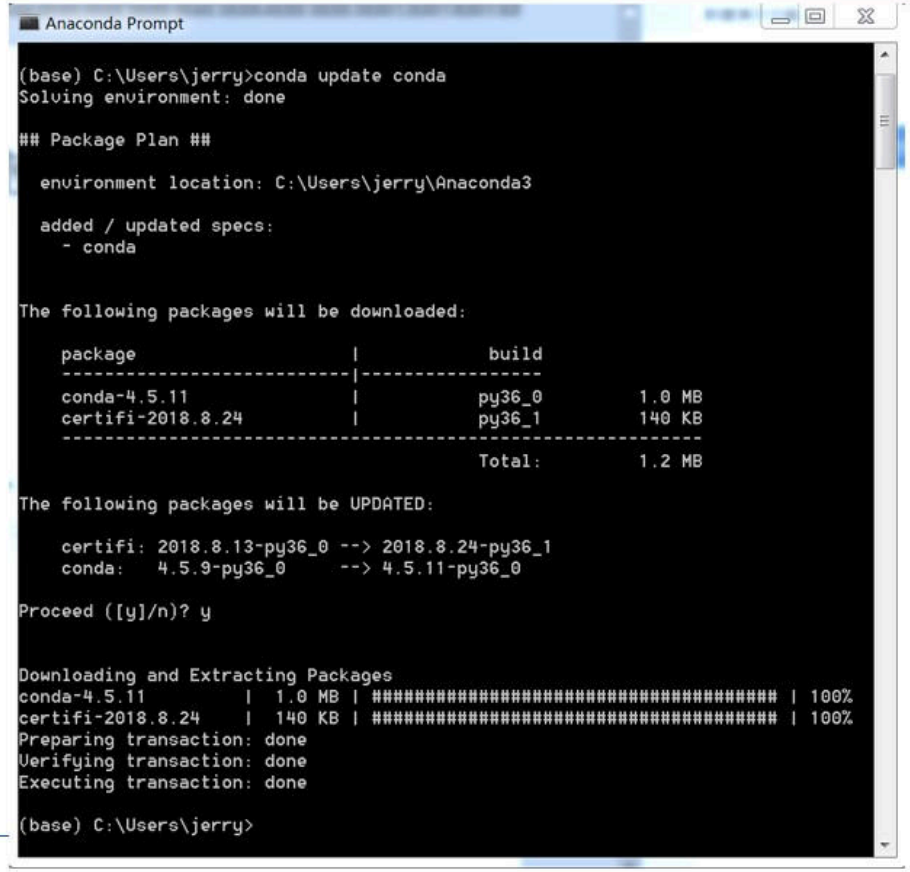
1. Verify that conda is installed

```
C:\WINDOWS\System32\Wind
(base) PS C:\Users\JLY> conda --version
conda 26.7.1
(base) PS C:\Users\JLY>
```

2. Update conda to the latest version

注意：如果出现错误信息，则需核实是否出现以下情况：

- 使用的用户是否是安装Anaconda时的账户。
- 是否在安装Anaconda之后重启了终端。



Using conda to manage the environment

1. Create a new environment (创建环境)

conda create --name <env_name> <package_names>

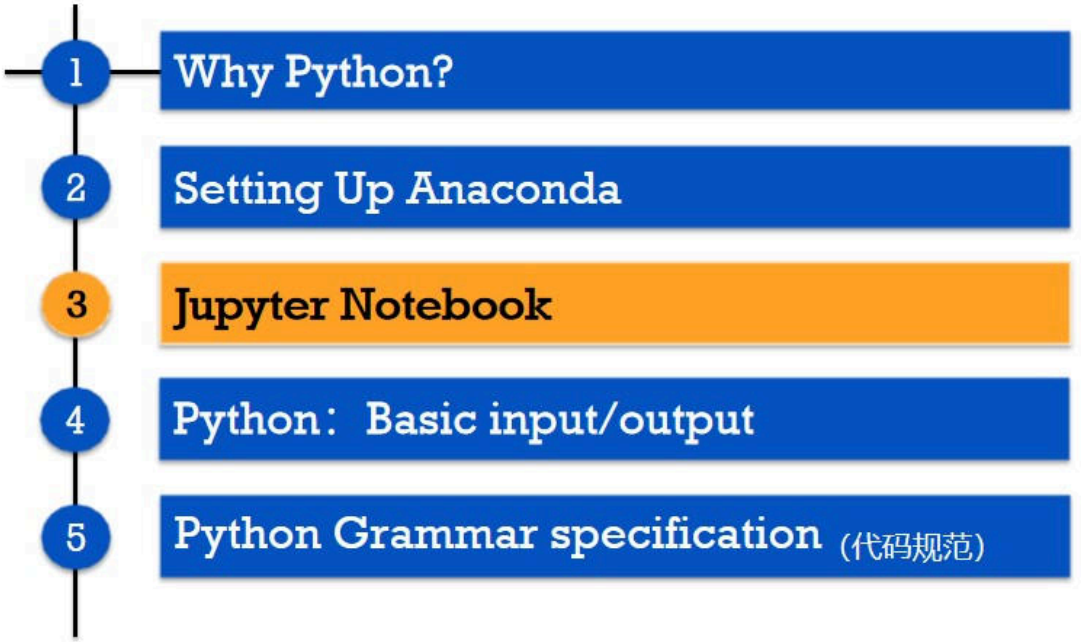
- 如果要安装指定的版本号，只需要在包名后面以=和版本号的形式执行，如果要在新创建的环境中创建多个包，则直接在<package_names>后以空格隔开，添加多个包名即可。
  - conda create -n python3 python=3.5 numpy pandas　#创建一个名为“python3”的环境，环境中安装版本为3.5的python，同时也安装了numpy和pandas这两个扩展包。

2. switch environment (切换环境)

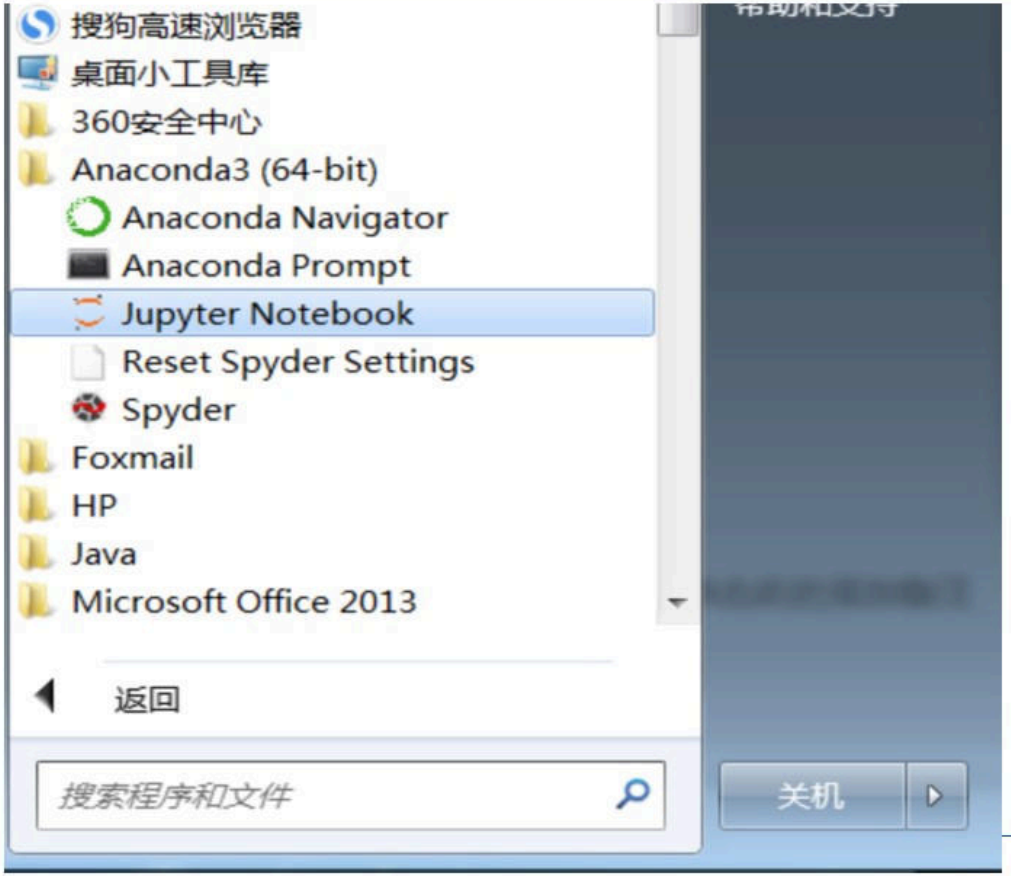
- activate <env_name>

Chapter 1: Getting Started

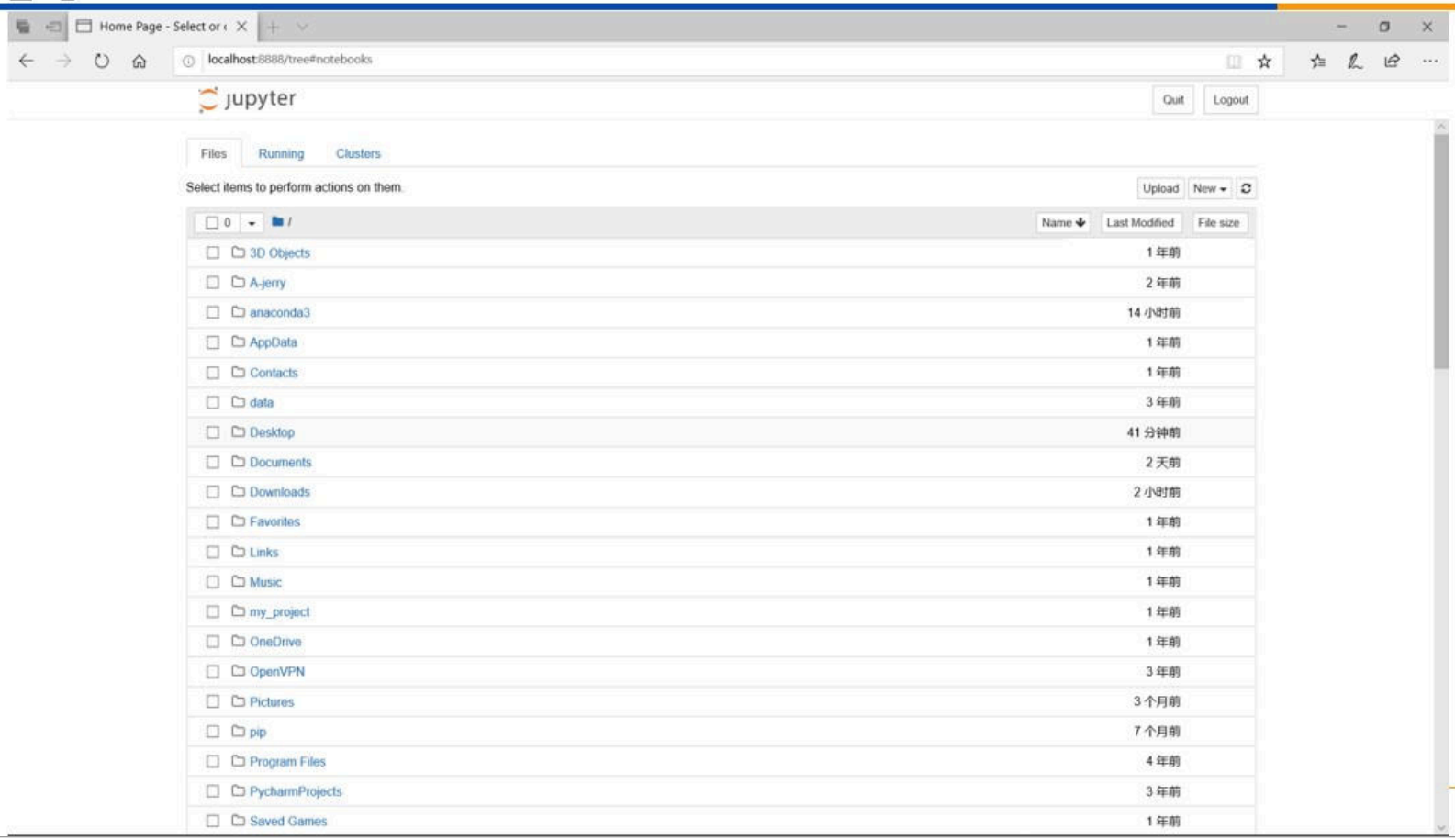
Contents



Jupyter Notebook



Jupyter Notebook



Jupyter Notebook

File Edit View Insert Cell Kernel Widgets Help Trusted Python 3

Run Markdown

数值型，字符串，逻辑向量

我们有四个专辑的发行时间 (1982, 1980, 1973, 1992) 我们想要将其放在一个名为 `Released_dates` 的变量里面。这意味着我们需要使用**set([ ])**创建一个向量。

使用数字，这变成一个**数值型向量。**

In [ ]:
```
1 Released_dates = [1982, 1980, 1973, 1992]
```

In [ ]:
```
1 Released_dates
```

如果我们使用引号呢?然后这就变成了**字符向量。**

In [ ]:
```
1 # 创建类型向量并为其赋值。
2 album_titles = ["Thriller", "Back in Black", "The Dark Side of the Moon","The Boduguard"]
3 album_titles
```

还有**逻辑向量**，它由True和False组成。当你想查看或对比内容时，它们是特别重要的。

In [2]:
```
1 ratings_dict = {"Thriller" : 10 , "Back in Black" : 8.5}
2
3 ratings_dict["Thriller"] > ratings_dict["Back in Black"]
```

Out[2]: True

Jupyter Notebook

File Edit View Insert Cell Kernel Widgets Help Trusted Python 3

Run Markdown

数值型，字符串，逻辑向量

```
1 我们有四个专辑的发行时间（1982，1980，1973，1992）我们想要将其放在一个名为`Released_dates`的变量里面。这意味着我们需要使用**set([ ])**创建一个向量。
2
3 使用数字，这变成一个**数值型向量**。
```

In [ ]:
```
1 Released_dates = [1982, 1980, 1973, 1992]
```

In [ ]:
```
1 Released_dates
```

如果我们使用引号呢?然后这就变成了**字符向量**。

In [ ]:
```
1 # 创建类型向量并为其赋值。
2 album_titles = ["Thriller", "Back in Black", "The Dark Side of the Moon","The Boduguard"]
3 album_titles
```

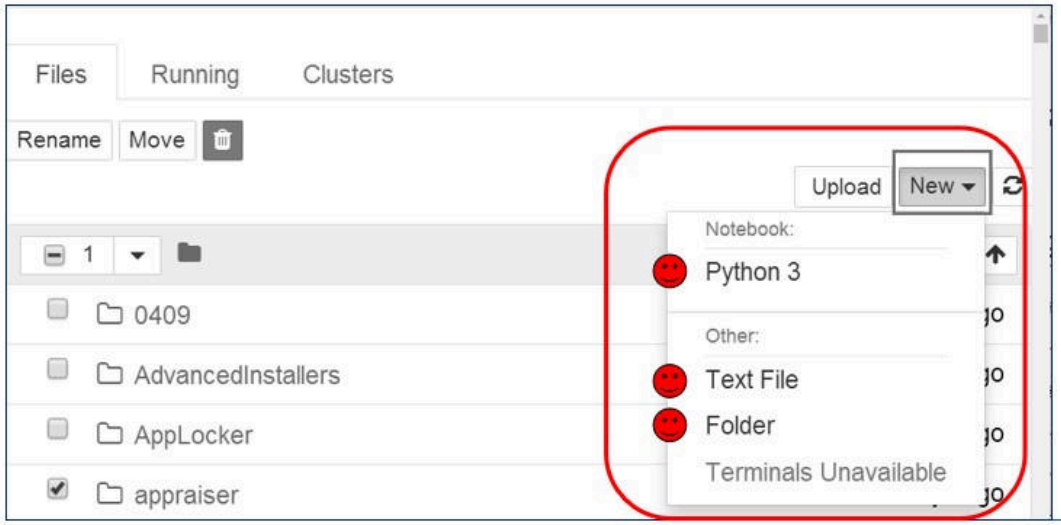
还有**逻辑向量**，它由True和False组成。当你想查看或对比内容时，它们是特别重要的。

In [2]:
```
1 ratings_dict = {"Thriller" : 10 , "Back in Black" : 8.5}
2
3 ratings_dict["Thriller"] > ratings_dict["Back in Black"]
```

Out[2]: True

Jupyter Notebook

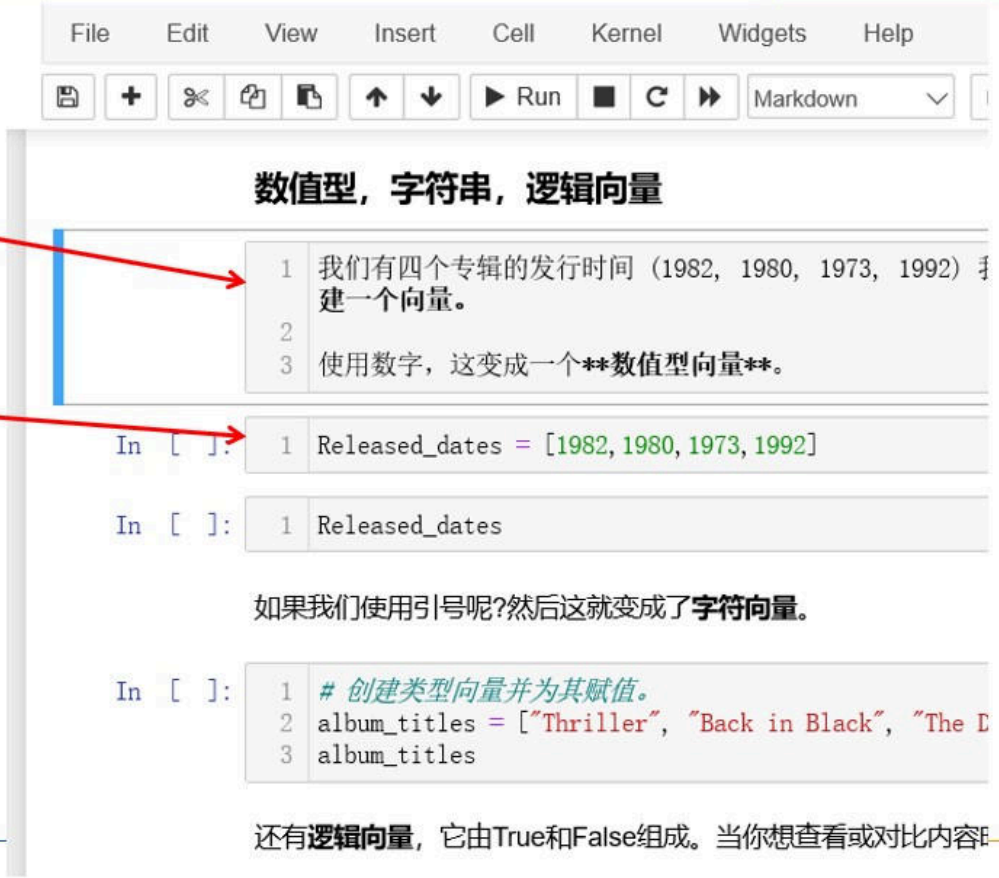
- Create and Open a new Notebook
  - "Text File" (纯文本型)
  - "Folder" (文件夹)
  - "Python 3" (表示 Python 运行脚本)



• 32

Jupyter Notebook

- **Markdown Cell:**
  - 在这里对文本进行编辑
- **code Cell:**
  - 这里是编写代码的地方



Jupyter Notebook

数值型，字符串，逻辑向量

```
1 我们有四个专辑的发行时间（1982, 1980, 1973, 1992）我们想要将其放在一个名为`Released_dates`的变量里面。这意味着我们需要使用**set([ ])**创建一个向量。
2
3 使用数字，这变成一个**数值型向量**。
```

编辑模式：用于编辑文本和代码

```
In [1]: 1 Released_dates = [1982, 1980, 1973, 1992]

In [2]: 1 Released_dates
```

数值型，字符串，逻辑向量

我们有四个专辑的发行时间 (1982, 1980, 1973, 1992) 我们想要将其放在一个名为 `Released_dates` 的变量里面。这意味着我们需要使用**set([ ])**创建一个向量。

使用数字，这变成一个**数值型向量**。

命令模式：用于执行键盘输入的快捷命令

```
In [1]: 1 Released_dates = [1982, 1980, 1973, 1992]

In [2]: 1 Released_dates

Out[2]: [1982, 1980, 1973, 1992]
```

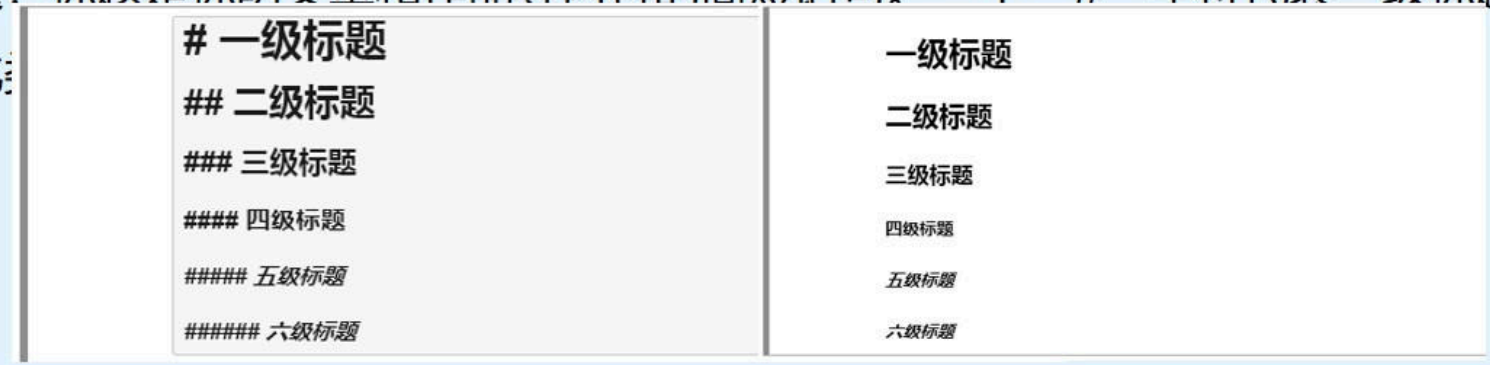

雨课堂 Rain Classroom

Jupyter Notebook Advanced Features（可略）

1. Markdown

Markdown 是一种可以使用普通文本编辑器编写的标记语言，通过简单的标记语法，它可以使普通文本内容具有一定的格式。

- 标题：标题是标明文章和作品等内容的简短语句。一个“#”字符代表一级标题，以此



Jupyter Notebook Advanced Features（可略）

1. Markdown

- 列表：列表是一种由数据项构成的有限序列，即按照一定的线性顺序排列而成的数据项的集合。

```
* Python
+ Python2
- Python3


1. Python
2. Python2
3. Python3
```

- Python
- Python2
- Python3

1. Python
2. Python2
3. Python3

- 对于无序列表，使用星号、加号或者减号作为列表标记
- 对于有序列表，则是使用数字“，”“（一个空格）”。

雨课堂 Rain Classroom

Jupyter Notebook Advanced Features（可略）

1．Markdown

- 加粗 / 斜体：前后有两个星号或下划线表示加粗，前后有 3 个星号或下划线表示斜体。

```
Python数据分析
**Python数据分析**
***Python数据分析***
__Python数据分析__
___python数据分析___
```

Python数据分析

Python数据分析

Python数据分析

Python数据分析

python数据分析

37

Jupyter Notebook Advanced Features（可略）

1. Markdown

- 表格：代码的第一行表示表头，第二行分隔表头和主体部分，从第三行开始，每一行代表一个表格行；列与列之间用符号“|”隔开，表格每一行的两边也要有符号“|”。

```
Python  | R| MATLAB |
------------ | ------------|----|
接口统一，学习曲线平缓 | 接口众多，学习曲线陡峭 |自由度大，学习曲线较为平缓 |
开源免费  | 开源免费 | 商业收费 |
```

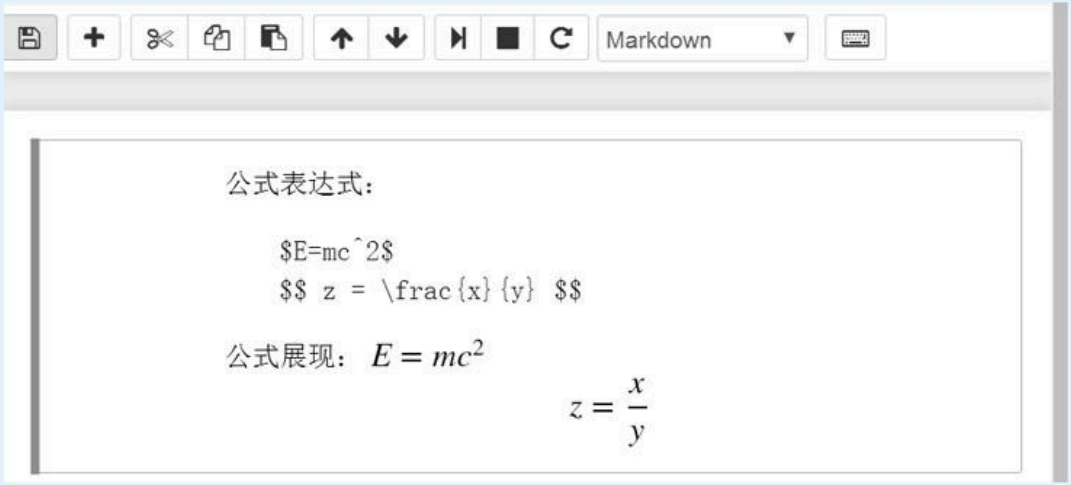
Python	R	MATLAB
接口统一，学习曲线平缓	接口众多，学习曲线陡峭	自由度大，学习曲线较为平缓
开源免费	开源免费	商业收费

雨课堂 Rain Classroom

Jupyter Notebook Advanced Features（可略）

1．Markdown

- 数学公式编辑：LaTeX 是写科研论文的必备工具， Markdown 单元中也可以使用 LaTeX 来插入数学公式。

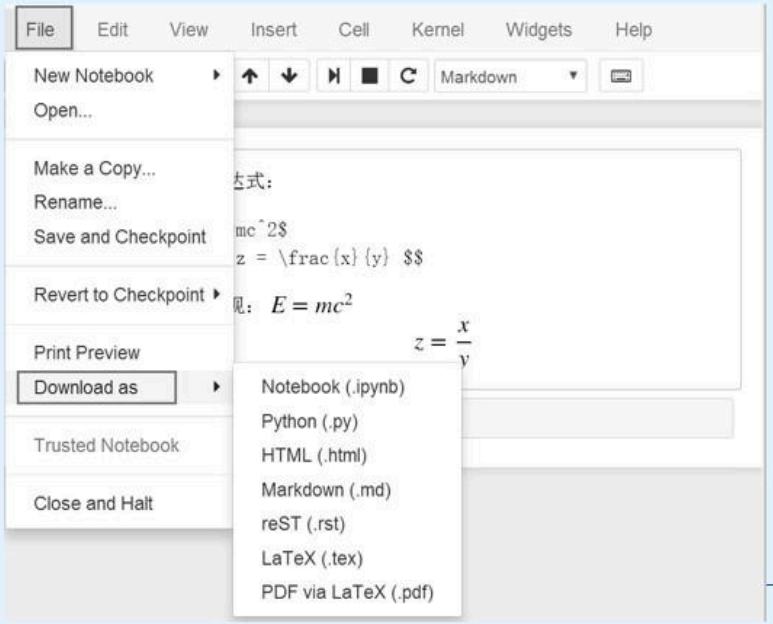


- 在文本行中插入数学公式，应在公式前后分别加上一个“$” 符号
- 如果要插入一个数学区块，则在公式前后分别加上两个“$$”符号。

Jupyter Notebook Advanced Features（可略）

2. Download as（导出功能）

- 可以将 Notebook 导出为多种格式，如HTML、Markdown、reST、PDF（通过 LaTeX）等格式。
- 导出功能可通过选择“File”→“Download as”级联菜单中的命令实现。



单选题 1分

Python的一个**主要**缺点是什么?

A 运行速度慢

B 代码不能加密

C 缩进导致混淆

D 不支持多线程

雨课堂 Rain Classroom

单选题 1分

Anaconda Distribution包含哪些组件?

A conda，Anaconda Navigator 和 Python

B Python和pip

C numpy和scipy

D Java和JavaScript

多选题 1分

关于环境管理，以下哪些选项是正确的?

A pip维护多个环境难度较大

B conda环境管理较为简单

C pip可以方便地在不同环境之间进行切换

D conda不会影响系统自带Python

雨课堂 Rain Classroom

多选题 1分

以下哪些是Jupyter Notebook的功能?

A 创建和打开新笔记本

B 编辑文本和代码

C 执行键盘输入的快捷命令

D 安装Python环境

雨课堂 Rain Classroom

单选题 1分

Markdown中如何表示一级标题?

A 使用一个星号

B 使用一个减号

C 使用一个“#”字符

D 使用一个数字

雨课堂 Rain Classroom

单选题 1分

关于Anaconda，以下说法正确的是：

A Anaconda是一个专门用于Web开发的Python框架

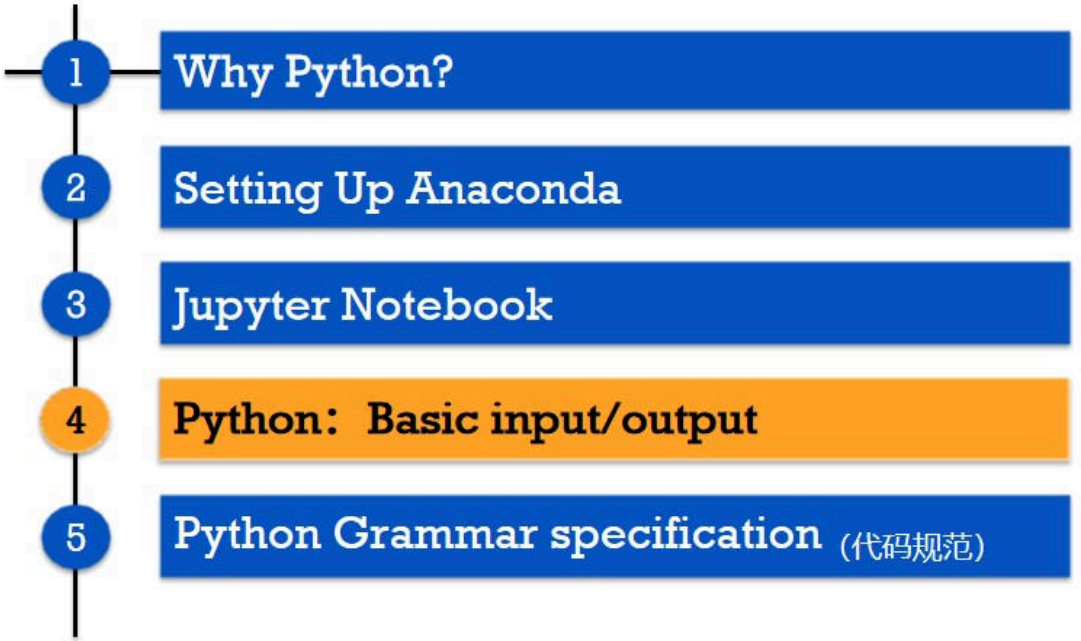
B Anaconda安装后只包含Python解释器，不包含任何第三方库

C Anaconda中的conda工具只能安装Python包，不能管理环境

D Anaconda是一个Python发行版，集成了数据科学常用的库和环境管理工具

Chapter 1: Getting Started

Contents



Python: Basic input/output

Input

- The general format of input( ) is:

```
a = input('提示: ')
```

```
(base) PS C:\Users\JLY> python
Python 3.14.7 | packaged by Anaconda, Inc. | (main, Aug 14 2026, 10:12:46)
[MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> a = input("input a number:")
input a number:12
```

```
>>> a
'12'
>>> type(a)
<class 'str'>
>>>
```

Python3 输入数字时，接收到的数据类型是str

输出a的值，可以看到输出的是一个字符串

雨课堂 Rain Classroom

Python: Basic input/output

Input

```
>>> a+1
Traceback (most recent call last):
  File "<python-input-3>", line 1, in <module>
    a+1
    ~^~
TypeError: can only concatenate str (not "int") to str
>>> int(a)+1
13
>>>
```

因为a是一个字符串，试图执行加法运算，所以出错

将字符串转为整数再执行加法运行，OK

```
# 读取整数
num = int(input())
# 读取小数
num = float(input())
```

如果需要输入整数或小数，则需使用int或float函数进行转换

雨课堂 Rain Classroom

Python: Basic input/output

Output

```
for i in range(10, 20):
    print(i, end=' ')
```

- The general format of print( ) is::

```
print([obj1,...][,sep=' '][,end='\n'][,file=sys.stdout])
```


雨课堂 Rain Classroom

Python: Basic input/output

Output

1. Omit all parameters（print函数所有参数均可省略）

```
>>>print()            # 无参数时，print函数输出一个空行
>>>
```

2. Output one or more objects

```
>>>print(123)         # 输出一个对象
123
>>>print(123,'abc',45,'book')     # 输出多个对象，对象之间默认空格分隔
123 abc 45 book
```

Python: Basic input/output

Output

3. Specify the output **delimiter**

```
>>>print(123,'abc',45,'book', sep='#')   # sep参数指定特定符号作为输出
123#abc#45#book                                    对象的分隔符号
```

4. Specifies the output **end symbol**

```
>>>print('price'); print(100)              #默认输出结尾，输出在两行
price
100
>>>print('price',end='='); print(100)     #指定输出结尾，输出在一行
price=100
```

Python: Basic input/output

Output

5. Output to file

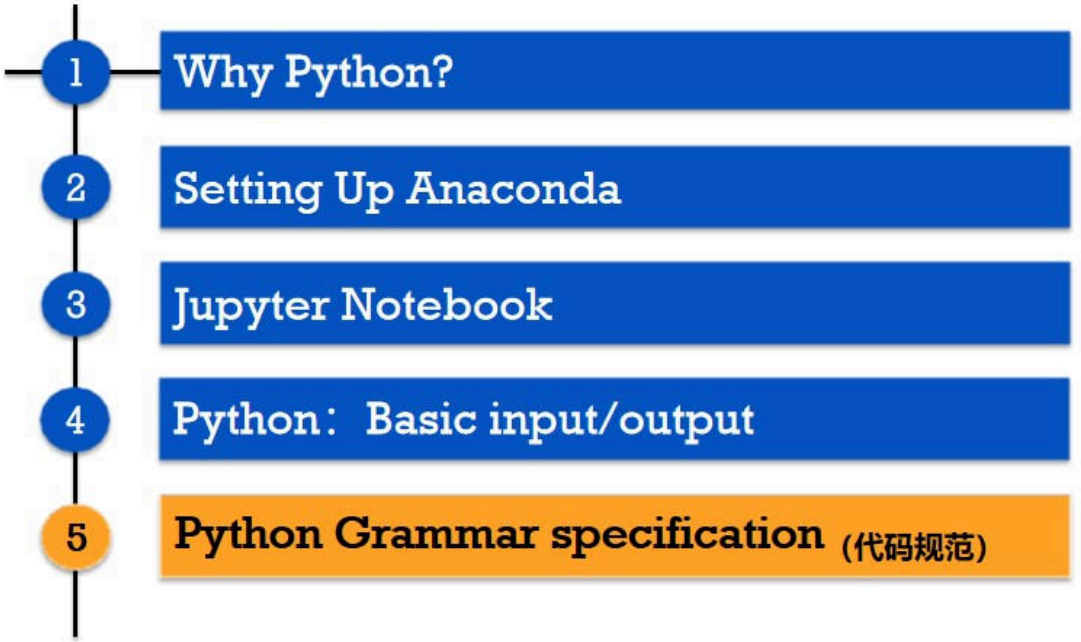
```
>>>file1 = open('data.txt','w')                    #打开文件
>>>print((123,'abc',45,'book',file=file1)          #用file参数指定输出到文件
>>>file1.close()                                   #关闭文件
>>>print((open('data.txt').read())                 #输出从文件中读出的内容
123 abc 45 book


>>>
```


雨课堂 Rain Classroom

Chapter 1: Getting Started

Contents



Python Grammar specification

(1) indentation 缩进

- Python uses indentation to determine line relationships in a program.

```
for i in range(10):
    print (i, end=' ')
```

magicians.py

```
magicians = ['alice', 'david', 'carolina']
for magician in magicians:
❶ print(magician)
```

```
  File "magicians.py", line 3
    print(magician)
        ^
IndentationError: expected an indented block
```

print 语句（见❶）应缩进却没有缩进。Python没有找到期望缩进的代码块时，会让你知道哪行代码有问题。

雨课堂 Rain Classroom

Python Grammar specification

(2) Comments 注释

- In Python, the hash mark (#) indicates a comment。

```
for i in range(10):        # 循环输出0到9数字（这是一个单行注释）
    print (i, end=' ')
```

comment.py

```
# Say hello to everyone.
print("Hello Python people!")
```

Python ignores the first line and executes the second line.

```
Hello Python people!
```


雨课堂 Rain Classroom

Python Grammar specification

(2) Comments 注释

```
def add(a,b):
    """This is a docstring indicating that this function is an additive function"""
    return a+b
```

```
In [1]: add.__doc__
Out[1]: 'This is a docstring indicating that this function is an additive function'
```

- 包含在一对三引号 '''...''' 或 """..."""之间且不属于任何语句的内容将被解释器认为是注释
  - 也叫文档字符串
- 文档字符串是包、模块、类或者函数里的第一个语句，可以通过对象的__doc__成员自动提取

雨课堂 Rain Classroom

Python Grammar specification

(3) import

```
>>>import math                          #导入math数学模块
>>>math.sin(0.5)                       #求0.5的正弦
>>>import random                        #导入random随机模块
>>>x = random.random()                  #获得[0,1]内的随机小数
>>>y = random.randint(1,100) #获得[1,100]上的随机整数
```

- **Order of import modules**
  1. **built-in** modules
  2. **third-party** modules
  3. modules **developed by yourself**

Python Grammar specification

(4) backslash (\)

```
>>>x = 'This is a very long very long very long very long very long very long \
... very long very long very long very long very long string'     # "\"来换行

>>>x = ('This is a very long very long very long very long very long very long'
... 'very long very long very long very long string')              # 圆括号中的行会连接起来
```


雨课堂 Rain Classroom

Python Grammar specification

(5) Necessary spaces and blank lines

- separate with spaces
  - both sides of the operator（运算符两侧）
  - between function parameters（函数参数之间）
  - both sides of comma（逗号两侧）

- Add blank lines for readability（可读性）
  - Between code blocks of different functions（代码块之间）
  - between different function definitions（函数定义之间）

雨课堂 Rain Classroom

Python Grammar specification

(6) keywords

- The following identifiers标识符 are used as reserved words, or keywords of the language, and cannot be used as ordinary identifiers.
- They must be spelled exactly as written here(including capitalization Python大小写敏感):

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	or	lambda try	
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

Using help

1. View help for built-in functions and types (查看内置函数和类型的帮助信息)

```
>>> help(max)
```

2. View member function information in a module (查看模块中的成员函数信息)

```
>>> import os
>>> help(os.fdopen)        # 查看os模块中的fdopen成员函数信息
```

3. View information for the entire module (查看整个模块的信息)

```
>>> import math            # 查看前，注意先import导入该模块
>>> help(math)             # 查看math整个模块的帮助信息
```


雨课堂 Rain Classroom

The Zen of Python (Python之禅)

```
(base) PS C:\Users\JLY> python
Python 3.14.7 | packaged by Anaconda, Inc. | (main, Aug 14 2026, 10:1
Type "help", "copyright", "credits" or "license" for more information
>>> import this
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to c
Although that way may not be obvious at first unless you're Dutc
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
>>>
```

```
>>> import this
The Zen of Python, by Tim Peters
Beautiful is better than ugly.
```

作者：Tim Peters（Python 早期核心开发者，Timsort 排序算法发明人），不是 Python 之父 Guido，但它提炼了 Guido 长期坚持的语言设计思想。19 条编程箴言，第20条让Guido补充，但至今未完成。

The Zen of Python

- **Beautiful is better than ugly.（优美胜于丑陋）**

  - Python程序员笃信代码可以编写得漂亮而优雅。编程是要解决问题的，设计良好、高效而漂亮的解决方案都会让程序员心生敬意。随着你对Python的认识越来越深入，并使用它来 编写越来越多的代码，有一天也许会有人站在你后面惊呼：“哇，代码编写得真是漂亮！”

```
# bad 丑陋：写法混乱、拼接杂乱
s=""
for i in [1,2,3]:
    s=s+str(i)+","

# good 优美：简洁清晰
s = ",".join(str(x) for x in [1,2,3])
```


雨课堂 Rain Classroom

The Zen of Python

- **Explicit is better than implicit.（明了胜于晦涩）**

```
# bad 晦涩，隐式类型转换，别人要猜逻辑
def calc(a,b):
    return a + b

# good 明了，标注参数含义
def add_numbers(x: int, y: int) -> int:
    return x + y
```

- def 定义函数；函数名 add_numbers，语义清晰：做数字相加。
- x: int, y: int：**类型注解**，冒号 : 后面写 int，代表建议参数 x, y 应当传入整数。类型注解只是提示，**不是强制检查**！Python 解释器不会强制拦截非 int 类型。
- -> int，**返回值注解**，代表：预期这个函数返回的结果是整数 int。**仅仅是标注，不强制。**

雨课堂 Rain Classroom

The Zen of Python

- **Simple is better than complex.（简洁胜于复杂）**

```
nums = [1,2,3,4]
# bad 过度复杂
res = []
for i in range(len(nums)):
    if nums[i]>2:
        res.append(nums[i])

# good 简单直白
res = [n for n in nums if n>2]
```

- 如果有两个解决方案，一个简单，一个复杂，但都行之有效，就选择简单的解决方案吧。这样，你编写的代码将更容易维护，你或他人以后改进这些代码时也会更容易。

雨课堂 Rain Classroom

The Zen of Python

- **Complex is better than complicated.（复杂胜于繁杂）**

  - 现实是复杂的，有时候可能没有简单的解决方案。在这种情况下，就选择最简单**可行**的解决方案吧。
  - 把业务逻辑、边界判断全部挤压在单行 lambda 里面。虽然代码很短能跑通，但可读性差；调试断点也不方便。lambda 适合简单逻辑，带 if-else 边界处理的业务逻辑，优先用普通 def 函数。

```
# good: 必要的复杂，逻辑分层清晰
def avg(data):
    if not data:
        return None
    return sum(data)/len(data)

# bad: 繁杂，全部揉在一行，极难读懂
avg = lambda d: sum(d)/len(d) if len(d)>0 else None
```

The Zen of Python

- **Flat is better than nested.（扁平胜于嵌套）**

```python
# bad 深度嵌套
def check(x):
    if x>0:
        if x<100:
            if x%2==0:
                return True
    return False

# good 扁平，提前return，减少嵌套层级
def check(x):
    if not 0 < x < 100:
        return False
    return x % 2 == 0
```

- 减少多层 if 嵌套（Python 缩进多了可读性暴跌）

The Zen of Python

- **Sparse is better than dense.（宽松胜于紧凑）**
  - 不要把一堆代码挤在同一行，留出空行分隔逻辑

```
# bad 高密度挤在一起
a=1;b=2;c=a+b;print(c)


# good 宽松，分行，逻辑分开
a = 1
b = 2
c = a + b
print(c)
```

The Zen of Python

- **Readability counts.（可读性至关重要）**
  - 可读性很重要，即便是复杂的代码，也要让它易于理解。开发的项目涉及复杂代码时，一定要为这些代码编写有益的注释。

```
# bad 变量无意义
x = [p*3 for p in arr if p>5]

# good 见名知意
triple_big_values = [num*3 for num in arr if num>5]
```

The Zen of Python

- **Special cases aren't special enough to break the rules.特殊情况也不足以打破规则**

  - 不要为了特殊场景随意搞一套特殊、不一致的写法

```
# bad: 普通数据用列表，遇到特殊数据就用魔法字符串，破坏统一规则
data = ["apple","banana"]
special_data = "NONE"

# good: 统一使用列表表达，特殊用空列表
data = ["apple","banana"]
special_data = []
```

The Zen of Python

- **Although practicality beats purity.（实用性胜过纯粹）**

  - 理想美学可以妥协，优先解决实际业务问题

```
# 理想：尽量不用全局变量；现实小脚本为快速完成任务，适度使用
g_total = 0
def add(n):
    global g_total
    g_total += n
```

The Zen of Python

- **Errors should never pass silently.（错误绝不应该被默默忽略）**

■ 不要随便吞掉异常，出错就暴露出来

```
# bad：把错误悄悄吃掉，出bug毫无线索
try:
    f = open("test.txt")
except:
    pass

# good：不随意掩盖错误，异常正常抛出
f = open("test.txt")
```

```
try:
    with open("test.txt") as f:
        data = f.read()
except FileNotFoundError:
    # 只捕获“文件不存在”这一种情况，明确处理这个问题
    print("文件没有找到")
```


雨课堂 Rain Classroom

The Zen of Python

- **Unless explicitly silenced.（除非明确选择忽略）**
  - 如果确实需要忽略异常，必须显式写明，告诉阅读者 “我就是故意忽略”

```
import os
# 明确写出：允许文件不存在的异常
try:
    os.remove("tmp.txt")
except FileNotFoundError:
    pass
```

The Zen of Python

- In the face of ambiguity, refuse the temptation to guess.（面对歧义，拒绝猜测的诱惑）
  - 不要猜输入是什么，要主动校验

```
# bad 猜测输入一定是数字
def square(val):
    return val*val

# good 不猜测，做判断校验
def square(val):
    if not isinstance(val,(int,float)):
        raise TypeError("必须传入数字")
    return val*val
```

The Zen of Python

- There should be one-- and preferably only one --obvious way to do it.（应当有一种，最好只有一种显而易见的做法。

- Python 设计理念：一件事尽量有标准唯一方案，而不是五花八门
- 如果让两名Python程序员去解决同一个问题，他们提供的解决方案应大致相同。这并不是说编程没有创意空间，而是恰恰相反！大部分编程工作都是使用常见解决方案来解决简单的小问题，但这些小问题都包含在更庞大、更有创意空间的项目中。
- 也就是说，在你的程序中，各种具体细节对其他Python程序员来说都应易于理解，也就是多用常规性做法，而不是标新立异。

```
# 拼接字符串，标准清晰方案 str.join()
items = ["a","b","c"]
text = " ".join(items)
```

· 76

雨课堂 Rain Classroom

The Zen of Python

- Although that way may not be obvious at first unless you're Dutch.尽管这种方式一开始未必显而易见——除非你是荷兰人。玩梗，调侃 Guido（荷兰人）。
- **Now is better than never.（当下动手胜过永远不做）**
  - 不要企图编写完美无缺的代码；先编写行之有效的代码，再决定是对其做进一步改进，还是转而去编写新代码。
  - 等你进入下一章，开始研究更复杂的主题时，务必牢记这种简约而清晰的理念。

```
# 先写出可用版本，后续迭代优化，不要一直空想不写
def calc_sum(lst):
    return sum(lst)
```

The Zen of Python

- **Although never is often better than *right* now.（但盲目仓促动手，还不如不做）**
  - 不要企图编写完美无缺的代码；先编写行之有效的代码，再决定是对其做进一步改进，还是转而去编写新代码。
  - 等你进入下一章，开始研究更复杂的主题时，务必牢记这种简约而清晰的理念。

```
# bad: 脑子一热随便写，不考虑边界
def avg(x):
    return sum(x)/len(x)

# 仓促写的代码没有处理空列表，直接写出来会直接崩溃，不如先想清楚再写
```

The Zen of Python

- If the implementation is hard to explain, it's a bad idea.
- If the implementation is easy to explain, it may be a good idea.

```
# bad: 晦涩难懂的一行魔法代码
res = lambda l: functools.reduce(lambda a,b:a+b,l)

# good: 逻辑直白，一眼看懂
def list_sum(lst):
    total = 0
    for item in lst:
        total += item
    return total
```

- 如果实现难以解释，那就是坏方案。
- 如果实现易于解释，那或许是好方案。

The Zen of Python

- Namespaces are one honking great idea -- let's do more of those!
  - 命名空间是绝佳的构想，多多善用!
  - 使用模块、类，利用命名空间隔离名字，避免变量名冲突

```
# mymath.py（模块，独立命名空间）
def add(a,b):
    return a+b

# main.py
import mymath
print(mymath.add(1,2))
```


雨课堂 Rain Classroom

单选题 1分

以下哪个选项是print()函数的基本格式?

A print([obj1,...][,sep=' '][,end='\n'][,file=sys.stdout])

B print([obj1,...][,delimiter=''][,end='\n'][,file=sys.stdout])

C print([obj1,...][,sep=' '][,end='\t'][,file=sys.stdout])

D print([obj1,...][,sep=' '][,end='\n'][,file=sys.stdin])

雨课堂 Rain Classroom

单选题 1分

在Python中，print函数的默认输出结尾是什么?

A 空行

B 空格

C 换行符

D 逗号

单选题 1分

Python使用缩进来确定代码行或代码块与程序其他部分的关系。以下哪个选项正确描述了缩进的作用?

A 缩进用于美化代码

B 缩进用于确定代码块的层次结构

C 缩进用于提高代码执行速度

D 缩进用于减少代码行数

雨课堂 Rain Classroom

单选题 1分

在Python中导入模块的正确顺序是?

A 第三方模块、内置模块、自定义模块

B 内置模块、第三方模块、自定义模块

C 自定义模块、内置模块、第三方模块

D 内置模块、自定义模块、第三方模块

雨课堂 Rain Classroom

多选题 1分

关于Python中的反斜杠（\）和圆括号，以下哪些说法是正确的?

A 反斜杠可以用来换行

B 圆括号中的行会自动连接起来

C 反斜杠和圆括号的功能是一样的

D 圆括号可以用来换行

雨课堂 Rain Classroom

单选题 1分

以下哪个不是Python的关键字?

A import

B class

C function

D yield

雨课堂 Rain Classroom

多选题 1分

使用help命令可以查看哪些信息?

A 内置函数的信息

B 模块的成员函数信息

C 整个模块的信息

D Python关键字的定义

雨课堂 Rain Classroom

单选题 1分

根据Python之禅，为什么简单的解决方案比复杂的解决方案更好?

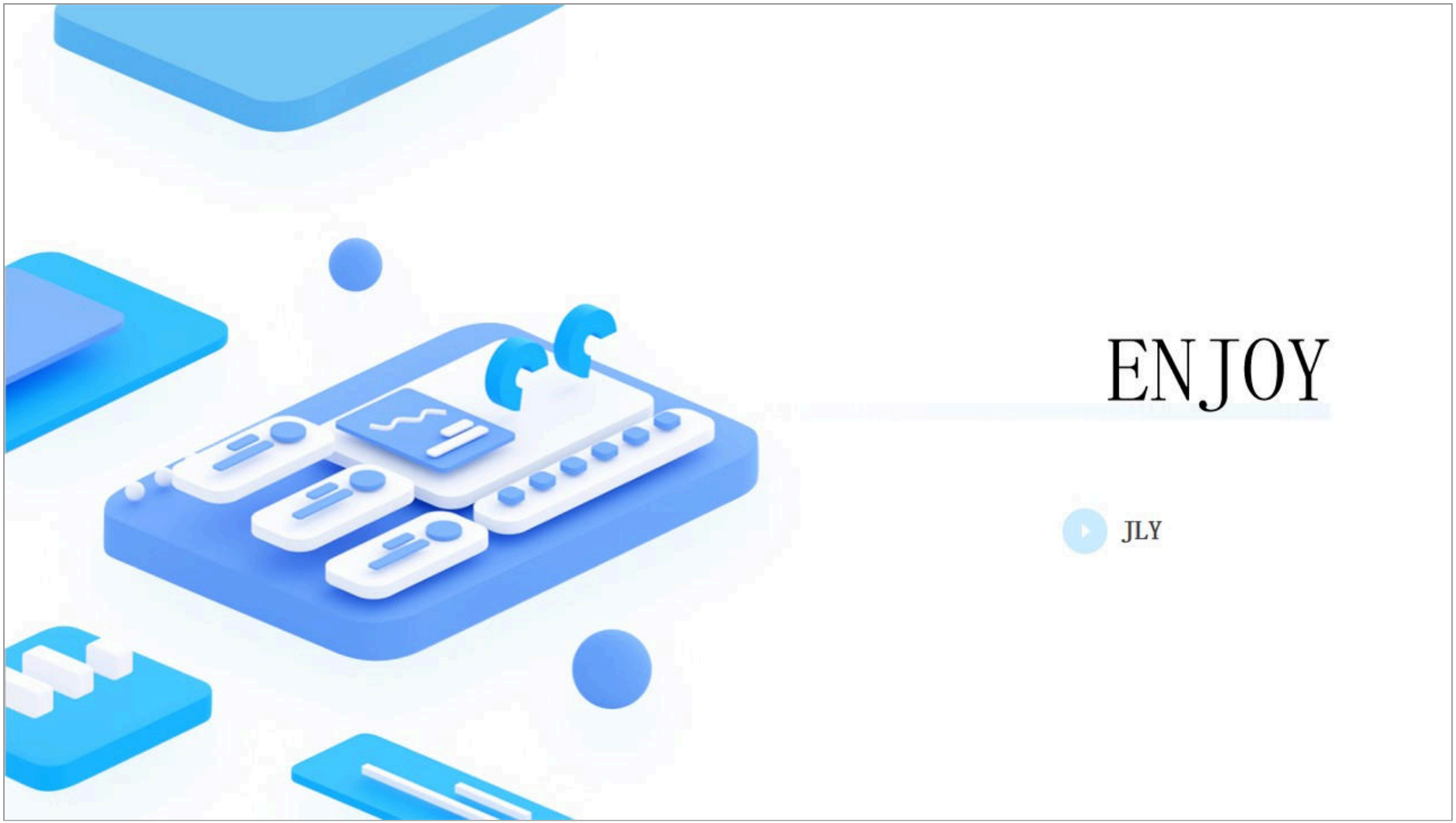
A 简单的解决方案更容易维护和改进

B 复杂的解决方案更具挑战性

C 复杂的解决方案更安全

D 简单的解决方案更快

雨课堂 Rain Classroom


ENJOY
JLY